Social Media Analytics Tool

Non-Relational Database Project

PRESENTED BY

Stephanie Borrego Arroyo

Hannah Chenoa Puente Rosales

Luis Fernando Del Real Vázquez

Teacher: Leobardo Ruiz Rountree | Date: 04/12/2025

Project Scope

A robust, scalable data architecture designed to monitor user activity, analyze complex social networks, and track engagement trends.

The system leverages the strengths of three distinct NoSQL databases plus a Vector Engine to handle diverse data types:

- **User Profiles:** Document-based storage.
- **Activity Logs:** High-throughput time-series.
- **Relationships:** Graph-based connection mapping.
- **AI Analytics:** Semantic search & sentiment analysis.

02. ARCHITECTURE

Polyglot Persistence Strategy

Each database was chosen to solve a specific data problem efficiently.

MongoDB

User Management & Profiles

Selected for flexible schema and JSON document mapping for user profiles.

Cassandra

Activity Logging

Selected for high write throughput and efficient time-series storage.

Dgraph

Relationship Tracking

Selected for native graph traversals (FOAF, Shortest Path).

ChromaDB

AI & Semantics

Selected for vector embeddings and semantic similarity search.

MongoDB Design & Indexing

Collection Separation

We utilized a **Two-Collection Pattern** to separate sensitive auth data from public profile data.

Index Justification

- **{ "email": 1 }**: Enforces uniqueness for auth.
- **{ "username": 1 }**: Fast public profile lookups.
- **{ "linked_accounts.platform": 1 }**: Optimization for aggregation pipelines analyzing platform popularity.

```
user_doc = {
    "_id": user_id,
    "username": username,
    "email": email,
    "password_hash": password_hash.decode(),
    "account_status": "active",
    "timestamps": {
        "created_at": datetime.utcnow(),
        "modified_at": datetime.utcnow()
    }
}

profile_doc = {
    "_id": user_id,
    "username": username,
    "full_name": full_name,
    "age": 25,
    "gender": "unknown",
    "bio": "",
    "profile_picture_url": None,
    "linked_social_accounts": []
}
```

Cassandra Time-Series

```
  CREATE TABLE IF NOT EXISTS interactions (
      user_id TEXT,
      post_id TEXT,
      interaction_type TEXT,
      timestamp TIMESTAMP,
      device_type TEXT,
      session_id TEXT,
      PRIMARY KEY (user_id, timestamp)
  ) WITH CLUSTERING ORDER BY (timestamp DESC);
")

  def get_interactions_by_user(self, user_id, limit=50):
      query = """
          SELECT * FROM interactions
          WHERE user_id = %s
          LIMIT %s;
      """
```

Wide-Column Design

Designed for **write-heavy** workloads where users generate constant streams of likes, comments, and shares.

Partitioning Strategy

- **Partition Key (user_id):** Groups all data for a single user together on the same node.
- **Clustering Key (timestamp):** Sorts data physically on disk in Descending order.
- **Benefit:** Allows retrieving the "Last 50 interactions" in O(1) time without sorting at read time.

Dgraph Network Analysis

Graph Schema

Unlike relational joins, Dgraph stores relationships as first-class citizens (Edges). We use **Facets** to store metadata directly on these edges.

Key Graph Query: Shortest Path

Finding the degrees of separation between users uses.

```
def create_schema(client):
    schema = """
        user_id: string @index(exact) .
        post_id: string @index(exact) .
        author_id: string .
        content: string .
        timestamp: datetime @index(hour) .
        relationship_strength: float @index(float) .
        interaction_type: string .
        follower_id: string .
        followed_id: string .

        FOLLOWS: [uid] @reverse .
        LIKED_POST: [uid] @reverse .
        COMMENTED_POST: [uid] @reverse .
        SHARED_POST: [uid] @reverse .
        POSTED_BY: [uid] @reverse .

        type User {
            user_id
            FOLLOWS
            LIKED_POST
            COMMENTED_POST
            SHARED_POST
            POSTED_BY
        }
        type Post {
            post_id
            author_id
            content
            POSTED_BY
            LIKED_POST
            COMMENTED_POST
            SHARED_POST
        }
        type Interaction {
            user_id
            post_id
            interaction_type
            timestamp
        }
```

Semantic Search & Sentiment

```
with col_chart:
    st.write("**Sentiment Distribution**")
    chart = (
        alt.Chart(df_sent)
        .mark_bar()
        .encode(
            x=alt.X("Sentiment", title="Sentiment Class"),
            y=alt.Y("count()", title="Number of Posts"),
            color=alt.Color(
                "Sentiment",
                scale=alt.Scale(
                    domain=["Positive 🟢", "Neutral ⚪", "Negative 🔴"],
                    range=["#2ecc71", "#95a5a6", "#e74c3c"],
                ),
            ),
            tooltip=["Sentiment", "count()", "mean(Score)"],
        )
        .interactive()
    )
    st.altair_chart(chart, use_container_width=True)

def query_database(query_text):

    results = collection.query(
        query_texts=[query_text],
        n_results=3
    )
    return results
```

Vector Embeddings

We use the all-MiniLM-L6-v2 model to convert text posts into high-dimensional vectors.

- **Semantic Search:** Finds relevance by meaning, not just keywords (e.g., "stress" matches "exams").
- **Sentiment Analysis:** Integrated with HuggingFace API to score brand health (Positive/Negative).
- **Keyword Extraction:** Analyzes clusters of vectors to find trending topics.

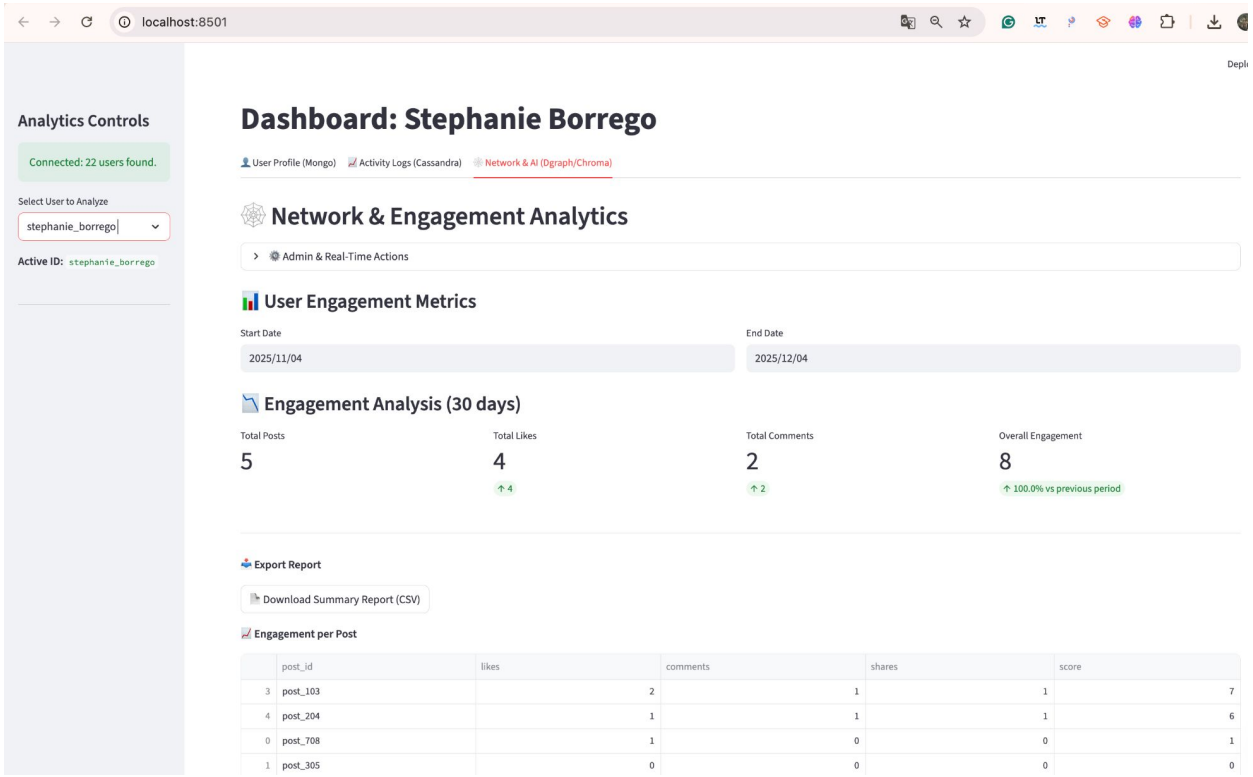
Unified Analytics Dashboard

Streamlit Integration

The frontend unifies all 4 data sources into a single view:

- **Real-time Logs:** Fetches from Cassandra.
- **Profile Editor:** Writes to MongoDB.
- **Graph Visualizer:** Renders Dgraph nodes using Graphviz.
- **AI Search:** Queries ChromaDB.

Feature Highlight: "Engagement Drop-off" calculates churn risk by comparing time-windows in Dgraph interactions.



08. WRAP UP

Conclusion

We successfully implemented a **NoSQL** architecture. By moving away from a SQL approach, we optimized each specific workload:

Scalability

Cassandra handles massive event streams without blocking user profile reads in Mongo.

Insight

Dgraph and ChromaDB unlock deep patterns (Influence & Sentiment) impossible in simple tables.